

مفاهیم زیر را تعریف کنید و ارتباط آن‌ها را با هم توضیح دهید:

DIP (Dependency Inversion Principle)

یکی از اصول SOLID است.

این اصل بیان می‌کند:

- ماژول‌های سطح بالا (Business Logic) نباید به ماژول‌های سطح پایین وابسته باشند.
- هر دو باید به Abstraction (Abstract Class یا Interface) وابسته باشند.
- جزئیات باید به Abstraction وابسته باشند، نه برعکس.

هدف:

کاهش وابستگی بین بخش‌های مختلف برنامه و افزایش قابلیت توسعه و تست.

IoC (Inversion of Control)

یک اصل طراحی است.

این اصل می‌گوید:

کلاس‌ها نباید مسئول ایجاد و مدیریت وابستگی‌های خود باشند؛

این مسئولیت باید به بخش دیگری واگذار شود.

به عبارت دیگر، کنترل ساخت و مدیریت وابستگی‌ها از کلاس خارج می‌شود.

فقط یک مفهوم است و نحوه پیاده‌سازی را مشخص نمی‌کند IoC.

Dependency Injection (DI)

یک تکنیک برای پیاده‌سازی IoC است.

در DI وابستگی‌ها از بیرون به کلاس تزریق می‌شوند،

به جای اینکه کلاس خودش آن‌ها را ایجاد کند.

انواع رایج:

- Constructor Injection
- Property Injection
- Method Injection

رابطه:

DI is a way to achieve IoC

IoC Container

ابزاری برای پیاده‌سازی خودکار Dependency Injection است.

وظایف:

- ساخت اشیا
- مدیریت وابستگی‌ها
- تزریق وابستگی‌ها
- مدیریت Lifetime سرویس‌ها

نمونه‌ها:

- Microsoft.Extensions.DependencyInjection
- Autofac
- Ninject

است DI از طریق IoC نیست، بلکه ابزاری برای پیاده سازی IoC خود IoC Container

Factory

یک الگوی طراحی (Creational Design Pattern) است که مسئولیت ساخت اشیا را برعهده می‌گیرد.

هدف:

جدا کردن منطق ساخت اشیا از منطق کسب‌وکار.

Factory as IoC

باشد IoC می‌تواند شکل ساده‌ای از Factory

زیرا مسئولیت ساخت اشیا را از کلاس اصلی گرفته و به کلاس Factory منتقل می‌کند.
اما:

واقعی نیست IoC Container یک Factory

زیرا معمولاً:

- وابستگی‌ها را به صورت خودکار Resolve نمی‌کند.
- مدیریت نمی‌کند Lifetime.
- را نمی‌سازد Dependency Graph.

به همین دلیل IoC Container را می‌توان Factory پیشرفته در نظر گرفت.

Service Lifetimes in .NET

مشخص می‌کنند IoC Container چه مدت یک شی را نگهداری کند.

Transient

هر بار که سرویس درخواست شود،
یک Instance جدید ساخته می‌شود.

service1 != service2

Scoped

در طول یک Scope فقط یک Instance وجود دارد.
در ASP.NET Core معمولاً منظور هر HTTP Request است.

service1 == service2

(در همان Request)

Singleton

فقط یک بار در کل طول عمر برنامه ساخته می‌شود
و همه از همان Instance استفاده می‌کنند.

service1 == service2

(در تمام برنامه)